

Notes written by: Yasar Baig

## Introduction To Python.

- What is Python? (Interpreter language):

Python is a programming language.

It is used to give instructions to a computer so it can do things like solving problems, building websites, analyzing data, making games or creating AI systems.

- Why Python is important?

- It is easy to read and write

- It is Beginner-Friendly

- Powerful and flexible

- What is Python history?

Python was created by Guido Van Rossum in 1989. It was officially released in 1991. He wanted to make language that is easy and fun to use.

- Where is Python used?

It is now used in many fields.

web development, data science, Machine learning, Automation, Game development, cybersecurity etc.

- Why Python is important?

easy to learn, used by big companies, many libraries, helps in modern field like AI, data science, Automation, Very huge community.

- What Kind of Python is there?

Python 3, CPython, Jython, PyPy, MicroPython.

## Print Statement In Python.

- The `Print()` function prints the given object to the standard output device. (screen) or to the text stream file.

`Print ("Mirza Yasin Abdullah Bala")`

It prints name or anything in terminal.

∴ The extension for Python file is `.py`.

`Print ( )` → Paratheses.

In Python Print are shown in line by line format.

Default output { Yasin  
24 in this way  
Bachkor

You can use single statement or ' ' .

`Print ('Yasin')`

If you want to show in 1 line

`Print ("Yasin")`

`Print ("Education", end = " ")`

`Print (24)`

You can print any data type using print

`print ("India", "Pakistan", "America")`

or

`Print ("Yasin", 24, 3.12, True)`

`Sep = ' ' / Automatically Provide space. You can use anything i.e ( /, -, -, \, % )`

## Comments In Python.

- In computer programming, comments are hints that we use to make our code more understandable.
- Comments are completely ignored by the Interpreter.
- Types of comments
  - Single line comments
  - Multi line comments.

```
Print ( "Yasir" ) * This is comment.
* Print ( "Balg" )
```

These are single line comments.

```
"""
```

```
Print ( """ This is a large
Paragraph """ )
```

```
"""
```

① Doc - Strings are also called double line comments.

## Escape Sequences.

Escape characters/sequences are illegal chars for Python & never get printed as part of output. When backslash is used in python programming, it allows the program to escape next characters.

- ' - Single quotation ("My name is 'Yasir'")
- " Double quotation ("My name is \"Yasir\"")
- \\ Backslash
- \n New Line
- \t Tab (use for space)
- \b Backspace.

## Variables In Python.

• Python variables are simply containers for storing data values. Unlike other languages such as Java, Python has no commands for declaring a variable, so you create one the moment you first assign a value to it.

```
x = 5  # x is variables.
```

```
Print (x)
```

```
name_Yasir = "Yasir"
```

```
Print (name_Yasir)
```

```
x = 2, y = 3, z = x + y
```

```
Print(z)
```

Rules for Variables.

a-z, A-Z valid

@bc → not valid

--- → valid

ab\_G → valid

\*123 → not valid

abc123 → valid

ab.cd → not valid

myVar → valid

Snake case first\_name

Method to Print variable

```
age = 24
```

```
Print ("my age is", age)
```

f-String

```
name = "Yasir"
```

```
Print(f"my name is {name}")
```

## Data types In Python

- Data Types specify what kind of value a variable holds. They also determine what operations (mathematical, relational, logical) can be performed on that value.

### Main Data Types In Python

→ Numerical Type → Int, float, complex

\* Integer (whole number)

```
num1 = 3
```

```
Print (num1)
```

\* Float (decimal numbers)

```
num2 = 3.14
```

```
Print (num2)
```

\* Complex number (real + imaginary)

```
num3 = 2 + 3j
```

```
Print (num3)
```

\* Boolean → True / False

```
is_active = True
```

```
Print (is_active)
```

\* String → Text enclosed in quotes

```
message = "Yasir"
```

```
Print (message)
```

\* List → Ordered, changeable, allows duplicates

```
fruits = ["apple", "banana", "orange"]
```

```
Print (fruits)
```

\* Tuple → ordered unchangeable (immutable)

```
coordinates = (10, 20)
```

```
Print (coordinates)
```

\* Set → Unordered, no duplicates

```
unique-num = {1, 2, 3, 4, 5}
```

```
Print {unique-num}
```

\* Dict - num = { "name": "Yasir" }

```
Print (Dic-num)
```

## Type Conversion / Type Casting

Type Conversion (or type casting) means changing the "data type" of a value.

• Python Provide built-in-functions like 'int()', 'float()', 'str()', 'list()', for type casting.

• It's useful when you need to perform operations that requires values to be in specific type.

• <sup>checking</sup> Changing the data type using type()

x = 4

Print(x, type(x)) \* output 4, int

• Type Conversion examples

x = int("100")

y = int("200")

z = x + y

Print(z) \* output 300

Summary of Type Casting functions in Py

int() — Converts to int (whole number)

str() — converts to str

float() — Converts to decimal number

list() — converts to list

dict() — converts to dictionary (valid format)

bool() — convert to True or False.

## Input Function In Python

The `input()` Function is used to take "user Input" whatever the user types is stored as a "string" by default.

• Simple example

```
x = input("Enter Your name: ")
Print("You entered:", x)
```

Do a Simple Example.

```
name = input("Enter name:")
gender = input("Enter gender:")
age = int(input("Enter age:"))
```

Display using F-Strings.

```
Print(f" My name is {name}, my age is {age}, and my gender is {gender}")
```

Note:

- All inputs are stored as string by default.
- use `int()`, `float()`, & `bool()` to convert them if needed.
- Example = `age = int(input("Enter number"))`

## Operators In Python

- What are Operators?

Operators are used to perform operations on variables and values.

- Different Type of Operators.

- Arithmetic operators
- Assignment operators
- Comparison operators
- Logical operators
- Identity operators
- Membership operators
- Bitwise operators.

- Arithmetic Operators.

Arithmetic operators are used with numeric values to perform mathematical operations.

- Addition (+)

`a = 10 + 5`

`Print (a)` ✖ output 15

- Subtraction (-)

`a = 10 - 5`

`Print (a)` ✖ output 5

- Multiplication (\*)

`a = 10 * 5`

`Print (a)` ✖ output 50

- Division (/)

`a = 10 / 5`

`Print (a)` ✖ output 2

- Modulus (%)

→ Modulus (%)

\* Modulus gives the remainder of division

$a = 10 \% 3$

Print(a) \* output 1

$b = 5 \% 10$

Print(b) \* output 5

$c = 9 \% 10$

Print(c) \* output 9

→ Exponential (\*\*)

used to raise a number to the power of another

$a = 5 ** 3$

Print(a) \* output 125

→ Floor Division (//)

It gives the result in whole numbers (Integer Parts only)

$a = 10 // 4$

Print(a) \* output 2

$a = -10 // 4$

Print(a) \* output -3

### • Assignment Operators

They are nothing, we already learn about them. Their main purpose is to assign value like

$a = 2 \Rightarrow a$  is assigned value (2)

Let's do some examples

$x = 5$

Print( $x$ )      ✖ output      5

$x += 5$

Print( $x$ )      ✖ output      10

$x = 5$

$x -= 2$

Print( $x$ )      ✖ output      3

$x = 5$

$x *= 3$

Print( $x$ )      ✖      15

$x = 5$

$x /= 2$

Print( $x$ )      ✖      2.5

$x = 5$

$x \text{ \% } 3 = 2$

Print( $x$ )      ✖ output      2

$x = 5$

$x \text{ // } 2$

Print( $x$ )      ✖ output

$x = 5$

$x ** 2$

Print( $x$ )      ✖ output

• Here is a table for Arithmetic Operators & for assignment operators  
∴ These Tables will build more understanding

## Arithmetic Table

| Operator | Name           | Example  |
|----------|----------------|----------|
| +        | Addition       | $x + y$  |
| -        | Subtraction    | $x - y$  |
| *        | Multiplication | $x * y$  |
| /        | Division       | $x / y$  |
| %        | Modulus        | $x \% y$ |
| **       | Exponentiation | $x ** y$ |
| //       | Floor Division | $x // y$ |

## Assignment Table

| Operators | Example    | Same As      |
|-----------|------------|--------------|
| =         | $x = 5$    | $x = 5$      |
| +=        | $x += 3$   | $x = x + 3$  |
| -=        | $x -= 3$   | $x = x - 3$  |
| *=        | $x *= 3$   | $x = x * 3$  |
| /=        | $x /= 3$   | $x = x / 3$  |
| %=        | $x \% = 3$ | $x = x \% 3$ |
| //=       | $x //= 3$  | $x = x // 3$ |
| **=       | $x ** = 3$ | $x = x ** 3$ |

## • Comparison Operators

Comparison operators are used to compare two values. The Result is always True / False.

$>$ ,  $<$ ,  $>=$ ,  $<=$ ,  $=$ ,  $!=$

These are some comparison operators we will understand with some examples & Tables.

| Operator | Comparison Name          | Operator Example |
|----------|--------------------------|------------------|
| >        | Greater than             | $x > y$          |
| <        | Less than                | $x < y$          |
| >=       | Greater than or equal to | $x \geq y$       |
| <=       | Less than or equal to    | $x \leq y$       |
| ==       | Equal                    | $x == y$         |
| !=       | Not equal                | $x != y$         |

- > Greater than  
Print (10 > 5) True
- < Less than  
Print (4 < 9) True
- ~~Print~~ Greater than or equal to  
Print (6 <= 3) False
- Greater than or equal to  
Print (7 >= 7) True
- equal to (==)  
Print (5 == 5) True
- Not equal to (!=)  
Print (8 != 10) True

These are some examples for comparisons.

### • Logical Operators.

Logical operators are used to combine multiple conditional statements

∴ Same we will discuss some examples and a table for better understanding

## Logical Operators

Operator

Description

Example

and

Return true if both true

$x < 10$  and  $y < 15$

or

return true if one is true

$x < 15$  or  $x < 24$

not

Reverse the result.

$\text{not}(x < 15 \text{ and } x < 20)$

$x = 10$

$y = 20$

1) and - Returns True only if both are True

$\text{Print}(x < 15 \text{ and } y < 25)$  \* True.

2) or - Returns True if one is True

$\text{Print}(x > 15 \text{ or } y > 15)$  \* True

3) not - Reverse the Result

$\text{Print}(\text{not}(x < 15 \text{ and } y < 25))$  \* False

AND Gate - Truth Table

| A | B | A & B |
|---|---|-------|
| 0 | 0 | 0     |
| 0 | 1 | 0     |
| 1 | 0 | 0     |
| 1 | 1 | 1     |

OR GATE - Truth Table

| A | B | A or B |
|---|---|--------|
| 0 | 0 | 0      |
| 0 | 1 | 1      |
| 1 | 0 | 1      |
| 1 | 1 | 1      |

## Conditional Statements

- What are conditional statements?  
(if, else, elif) allow us to control the flow of program. They help make decision based on conditions.

### • Type of Conditional Statements

- If statement
- else statement
- elif statement
- nested if-else

★ Example : check person eligible to vote

```
age = int(input("Enter your age: "))
```

```
if age >= 18:
```

```
    print("You can vote")
```

```
else:
```

```
    print("You can't vote")
```

★ Example No 2 " If else, elif.

```
num1 = int(input("Enter Num1"))
```

```
num2 = int(input("Enter Num2"))
```

```
if num1 > num2:
```

```
    print("Num 1 is greater")
```

```
elif num1 < num2:
```

```
    print("Num 2 is greater")
```

```
else:
```

```
    print("Both are equal")
```

∴ You can add many elif if your program wants.

# Loops In Python

- What are loops?

Looping means representing a block of code multiple times until a particular condition is satisfied.

- Main Type of loops In Python.

- while loop
- For Loop

while loop Syntax

```
while condition :  
    code  
    code
```

It will keep executing the block as long as the condition is True.

Print("It prints, when condition become false")

Example: (Print hello world 10 Time)

```
i = 1
```

```
while i < 10 :
```

```
    Print ("Hello world")
```

```
    i = i + 1
```

Example: (Print even number from 1 to 20)

```
i = 1
```

```
while i <= 20 :
```

```
    if i % 2 == 0 :
```

```
        Print (i, end = " ")
```

```
    i = i + 1
```

For loop syntax  
for variable in range (start, stop, step):

Example:

```
for i in range (1, 20):  
    Print(i)
```

Example (only odd)

```
for i in range (1, 20, 2):  
    Print(i)
```

Example: Reverse Value

```
for i in range (10, 0, -1):  
    Print(i)
```

Diff

For loop

- Syntax is easy
- You know exactly how many time to run loop
- Iterating over sequence

While loop

- Syntax is difficult
- Don't know in advanced how many time to in loop
- You want to run until condition is false

Nested loop Example:

```
for i in range (1, 4):  
    Print ("outer loop i = ", i)  
    for j in range (10, 14):  
        Print ("inner loop j = ", j)
```

## Control Statements Python

• What are Control statements?

These statements are an essential aspect of any programming language. They are used to manage the flow of execution in a program based on certain conditions.

Python Control Statements.

- Break
- Continue
- Pass

Break Statements.

The break statements used to terminate the loop immediately when certain condition is met.

Example:

```
for i in range(1, 11):
```

```
    print(i)
```

```
    if i == 5:
```

```
        break
```

\* output = 1, 2, 3, 4, 5

Continue statements:

It skip the current iteration & move to next one

Example

```
for i in range(1, 11):
```

```
    if i == 5:
```

```
        continue
```

\* skip 5

```
    print(i)
```

\* 1, 2, 3, 4, 6, 7, 8, 9, 10

Pass Statement

It does nothing (used as placeholder for future code).

## Lists In Python

What are lists?

Lists are sequence data types in Python that can store multiple values in a single variable.

Lists are mutable (modifiable) and can hold elements of different type (int, float, str). They are similar to array but not flexible.

\* Creating an empty list

```
my_list = []
```

\* Creating a list with mixed data types

```
list = [1, 2, 3, "Yasir", True, 0.7]
```

```
Print(list) * [1, 2, 3, "Yasir", True, 0.7]
```

\* Accessing Elements From a list

```
x = [45, 21, 41, 2, 8, 1, 6]
```

```
Print(x[2]) * 2
```

```
Print(len(x)) * 7
```

```
Print(x[-1]) * 3
```

```
Print(type(x[2])) * <class 'int'>
```

\* Iterating Through a list

1. By Index

```
a = [1, 2, 43, 54, 5, 5, 6, 67, 4]
```

```
for i in range(len(a)):
```

```
Print(a[i])
```

2- By value (cleaner)

for value in a:

Print (value)

Built in Functions in List

a = [1, 2, 4, 55, 6, 6, 6]

Print (len(a))

Print (sum(a))

Print (max(a))

Print (min(a))

\* List Methods

my-list = [10, 20, 30, 40]

1- Append - Add items to the end

my-list.append(50)

Print(my-list)

2- Insert - Insert at position i

my-list.insert(2, 25) - then print

3- extend (iterable) - Add multiple values.

my-list.extend([60, 70]) then print.

4- remove - Remove first value

my-list.remove(25)

5 Pop - Removes & return items at index.

my-list.pop()

6. Index - Return first index

```
Print (my-list.index(30))
```

7. Count (x) - Counts occurrences of x

```
Print (my-list.count(10))
```

8. Sort() - Sort in ascending order

```
num-list = [5, 2, 9, 1, 7]
```

```
num-list.sort()
```

9. Reverse() Reverse the list

```
num-list.reverse()
```

```
Print (num-list)
```

10. Copy() Returns a shallow copy

```
copy-list = num-list.copy()
```

```
Print (copy-list)
```

11. Clear - Empty the lists

```
copied-list.clear()
```

```
Print (copied-list)
```

List slicing

\* What is slicing?

Slicing means extracting a part of the list using syntax:  
list [start:end:step]

```
nums = [10, 20, 30, 40, 50, 60, 70, 80, 90, 100]
```

### \* Basic Slicing

Print (nums [0:5])

Print (nums [:4])

Print (nums [5:])

### \* Step Slicing

Print (nums [::2])

Print (nums [1::2])

### \* Negative slicing (Reverse)

Print (nums [::-1])

Print (nums [-3:])

Print (nums [-5:-2])

### \* Why use slicing?

For data cleaning, data extraction, reversing lists, skipping elements, or efficient subsetting

last lines for list

- They are essential for managing & manipulating sequence of data.

- Use method like append, pop to modify them

- Use slicing for powerful subsetting & Transformations.

## Tuples In Python

\* What are tuples in Python?

Tuples are similar to list, but with one key difference. Tuples are immutable once created. they can't be changed.

Tuples supports indexing, slicing, iteration & nesting.

They have only 2 methods, count/index.

### Creating tuple

```
tuple1 = ()
```

```
Print (tuple1)
```

```
tuple2 = (1, 2, 3, 4)
```

```
Print (tuple2)
```

```
tuple3 = (1, 2, 3, "Yasir", True)
```

```
Print (tuple3)
```

Single Item tuple - comma is mandatory

```
tuple4 = (1,)
```

```
Print (tuple4)
```

Creating tuple from string

```
tuple5 = tuple("Yasir")
```

Creating Tuples From List

```
tuple6 = tuple([1, 2, 3, 4, 5])
```

```
Print (tuple6)
```

### Accessing Tuple Elements

```
tuple11 = (1, 2, 3, 4, 5, 6)
```

```
Print (tuple11[1])
```

```
Print (tuple11[-2])
```

### Nested Tuple

```
tuple22 = (1, 2, 3, (4, 5))
```

```
Print (tuple22[-1][0])
```

### Tuple Immutability

```
deltuple = (1, 2, 3, 4, 5)
```

```
Print (deltuple)
```

Tuple Operations :  $tuple1 = (1, 2)$ ,  $tuple2 = (3, 4)$

- Concatenation

```
Print (tuple1 + tuple2)
```

- Repetition

```
Print (tuple1 * 2)
```

```
Print (tuple2 * 3)
```

### Looping Through Tuple

```
Print ("Elements in tuple1:")
```

```
for i in tuple1:
```

```
    Print(i)
```

### Membership Testing

```
Print ("Is 1 in tuple1?", 1 in tuple1)
```

```
Print ("Is 1 not in tuple1?", 1 not in tuple1)
```

## • Tuples Built-in-Functions

function\_tuple = (1, 2, 2, 4, 54, 65, 6)

```
Print ( " Length:", len(function_tuple) )
Print ( " Minimum Value:", minlen(function_tuple) )
Print ( " Maximum Value:", maxlen(function_tuple) )
Print ( " Sorted (ascending):" maxsorted(function_tuple) )
Print ( " Sorted (descending):" sorted(function_tuple, reverse
= True) " Return a list.
```

## Important Points for

- Tuples are immutable
- Tuples are only readable
- Tuple use less values
- Tuples are faster for fixed data.
- Good for protecting data Integrity

## Strings In Python.

\* What are strings in Python?

Strings are sequence of uni-code characters

Example: "Yasir" = 'Y', 'a', 's', 'i', 'r'

• Creating strings

a = "Yasir Baig" - Double Quotes

Print(a)

b = 'Yasir Baig' - Single quotes

Print(b)

c = "It's raining" - Double quotes handle single.

Print(c)

d = """Yasir is an AI developer"""

Print(d) - Triple quotes.

e = str("Mirza Yasir Abdullah Baig")

Print(e) - using str - constructor.

• Accessing String characters (Indexing)

name = "Yasir"

Print(name)

Positive Indexing

Print(name[0])      ✖    Y

Print(name[1])      ✖    a

Print(name[2])      ✖    s

Print(name[3])      ✖    i

Print(name[4])      ✖    r

Print(name[5])      ✖ Error Index out of range

## Negative Indexing

```
Print (name [-1])    *    r
Print (name [-2])    *    l
Print (name [-3])    *    s
Print (name [-4])    *    a
Print (name [-5])    *    Y
```

## • Slicing Strings

```
text = "Mirza Yasin Abdullah Bary"
```

```
Print (text [0:6])    * "Mirza "
Print (text [2:])     * "rza Yasin Abdullah Bary"
Print (text [:5])     * "Mirza"
Print (text [:])      * Full String
Print (text [0:6:2])  * "Mra"
Print (text [-6:-1:2]) * " B"
Print (text [::-1])   * Reversed String
Print (text [-1:-7:-1]) * Last 6 char reversed
```

## • Editing & Deleting string

```
s = "AI Machine Learning Journey"
Print (s)
```

## • Reassigning is allowed

```
s = AI course
```

```
Print (s)
```

- Deleting specific character not allowed.
  - Entire string deletion is allowed.
- ```
del s
```

## • Strings Operators

### • Arithmetic

Str1 = "Yasir"

Str2 = "Baig"

Print (Str1 + " " + Str2) ✖ Concatenation

Print (Str1 \* 3) ✖ Repetition.

Print (" " \* 40)

### ✖ Relational Operators

Print ("Yasir" == "Baig")

Print ("Abdullah" != "Mirza")

Print ("Islamabad" > "Kasur")

### • Logical Operators

Print ("Yasir" and "Baig") - 'Baig'

Print ("Yasir" or "Baig") - 'Yasir'

Print (" " and "Baig") - falsy

Print (" " or "Baig") - 'Baig'

### • Looping In Strings

for char in "Mirza Yasir Abdullah Baig"

Print (char)

### • Membership Operators

S = "Mirza Yasir Abdullah Baig"

Print ('M' in S) ✖ True

Print ('Z' in not in S) ✖ True

## • Common Strings Functions.

```
str_function = "Mirza Yasir Abdullah Baig"
Print ( len (str_function))
Print ( max (str_function))
Print ( min (str_function))
Print ( sorted(str_function))
Print ( sorted (str_function, reverse = True))
```

## • String Specific Methods.

```
example = "mirza yasir abdullah Baig"
Print ( example . capitalize())
Print ( example . title())
Print ( example . upper())
Print ( example . lower())
Print ( example . swapcase())
Print ( example . count("a"))
Print ( example . find("r"))
Print ( example . startswith("mirza"))
Print ( example . endswith("Baig"))
```

Split & Join  $\Rightarrow$  line = "Mirza Yasir Baig"

```
print ( line . split())
Print ( line . split(" "))
Print ( line . split("x"))
```

Join  $\rightarrow$  names = ["Mirza", "Yasir", "Abdullah"]

```
Print ( "- " . join (names))
```

Replace & strip (You must do)

## Dictionary In Python.

- What is dictionary?

A dictionary is the unordered, mutable & indexed collection. It stores data in key & value pairs.

Syntax = { key : value }

Why do we use dictionary?

- Fast data retrieval using keys
- Meaningful key-value mapping
- Efficient data organization.

- Creating Dictionaries.

```
empty-dic = { }
```

```
Print (empty-dic)
```

```
Simple-dic = { "name": "Yasir" }
```

```
Print (Simple-dic)
```

Using dict constructor

```
user = dict ( name = "Yasir", age = 30 )
```

```
Print (user)
```

- Accessing Items

```
Print ( simple-dic [ "name" ] )
```

```
Print ( simple-dic . get ( "role" ) )
```

```
Print ( simple-dic . get ( "salary", 0 ) )
```

- Adding & updating key-value pairs.

```
simple-dict [ "experience" ] = 2
```

```
Simple-dict [ "age" ] = 25
```

```
Print ( simple-dict )
```

- Removing Key-value Pairs.

```
del simple-dict ["role"]
```

```
Print (simple-dict)
```

```
removed = simple-dict.pop("experience")
Print (removed)
```

- Iterating Over dictionary

```
data = { "name": "Yasir", "field": "AI",
         "level": "Advanced" }
```

✧ Keys

```
for key in data:
```

```
    Print (key)
```

Value

```
for value in data.values():
```

```
    Print (value)
```

Key - value Pairs

```
for key, value in data.items():
```

```
    Print (f" {key} => {value} ")
```

- This is all about dictionary

Dictionaries are very useful in Production level work.

Dictionaries are important & for Placement the concept is very necessary.

## Sets In Python

- What are Sets?
- Sets are unordered collections of unique, immutable elements.
- They don't allow duplicates.
- They are mutable
- They don't support indexing / slicing.

### • Creating Sets.

`S1 = { }` - wrong way  
`Print (type(S1))`  $\neq$  dict.

correct empty set.

`S2 = set()` ~~`set()`~~  
`Print (type(S2))`  $\neq$  Set.

### $\neq$ Creating a set with duplicates

`S3 = { 1, 2, 3, 3, 4, 4, 5 }`  
`Print (S3)` - `{ 1, 2, 3, 4, 5 }`

$\rightarrow$  Sets inside sets are not allowed.

### • Type conversion

`list_to_set = set([1, 2, 2, 3])`  
`Print (list_to_set)`  $\neq$  `{ 1, 2, 3 }`

### • Modifying Sets.

`S = { 1, 2, 3 }`  
`S.add(4)`  
`Print(S)`

s.update([5, 6])

Print(s)

s.remove(2)

Print(s)

s.discard(10)

Print(s)

s.pop()

Print(s)

s.clear()

Print(s)

### • Membership Operators.

member = { "Yasir", "abdullah", "Baig" }

Print("Yasir" in member) \* True

Print("Ali" not in member) \* True

### • Looping Through Sets.

loop\_set = { 1, 2, 3, 4, 5 }

for item in loop\_set:

Print(item)

### • Set Methods

A = { 1, 2, 3 }

B = { 3, 4, 5 }

Print(A.union(B))

Print(A | B)

Print(A.intersection(B))

Print(A & B)

Print (A.difference(B))

Print (A - B)

Print (A.symmetric\_difference(B))

Print (A ^ B)

### • Set Relationships.

set1 = {1, 2}

set2 = {1, 2, 3}

Print (set1.issubset(set2))      ✗ True

Print (set2.issuperset(set1))      ✗ True

Print (set1.isdisjoint({4, 5}))      ✗ True.

### • Built-In Functions

nums = {1, 2, 3, 4, 5}

Print (len(nums))

Print (max(nums))

Print (min(nums))

Print (sum(nums))

Print (sorted(nums))

• Sets are ideal for membership testing, uniqueness filtering, & mathematical set operations.

• Use FrozenSet if you need an immutable version of a set.

• Set are unordered; their printed order may vary.

## Functions In Python

- What are Functions, why use Functions?
- Function helps

- Organize code into blocks
- Avoid repetition
- Make code modular, reusable, readable
- Perform specific task independently.

Syntax of a Function.

```
def func(Parameters):
    """ docstring """
    # Func Body
    return value
```

function - call.

```
def greet():
    Print ("hello, how are you")
greet()
```

Return vs Prints.

- Print - display output to console
- return - send data back to caller for further use.

Parameters vs Arguments.

- Parameters: Variable inside function
- Arguments: actual value pass when calling func

Types of Parameters

- Required Parameters.

```
def required_example(name):
    Print (name)
required_example("Yasir")
```

## 2- Default Parameters

```
def default_example(city = "Lahore")
    Print (city)
default_example()
default_example("Karachi") // overrides Lahore.
```

## 3- Named Parameters

```
def student_info(name, age):
    Print (name)
    Print (age)
student_info(age = 24, name = "Yasir")
```

## 4- Variable-length Parameters

```
def many_args(*args):
    Print (args)
many_args(1, 2, 3, "Python")

def many_kwargs(**kwargs):
    Print (kwargs)
many_kwargs(name = Yasir, age = 24)
```

## • Scope In Functions

- Local scope: variables declared inside func
- Global scope: variables declared outside all func

## File handling In Python

• What is File handling?

File handling in python refers to the ability to read, write, create, delete, & manipulate files stored on your computer (like .txt, .csv, .json, files)

• Why we use File handling?

- To store data permanently.
- To read logs, input, config files
- To work with datasets.

### File Modes.

|    |   |              |
|----|---|--------------|
| r  | - | read         |
| w  | - | write        |
| a  | - | append       |
| x  | - | create       |
| rt | - | read & write |
| b  | - | binary mode  |

• Exception handling (try, except)

try:

```
with open("nofile.txt", "r") as f:
```

```
    content = f.read()
```

```
except FileNotFoundError:
```

```
    Print ("File not Found")
```

```
except PermissionError:
```

```
    Print ("Permission denied")
```

```
else:
```

```
    Print ("File read successfully")
```

```
finally:
```

```
    Print ("operation completed.")
```

## Read Methods.

- `read()`
- `readline()`
- `readlines()`

## Important Tips.

- Always use `'with open(...)'` - auto-close file safely.
- use exception handling to prevent crash.
- File Paths. use relative (`./folder/file.txt`)
- Always close file if not using `'with'`

"The End"

"Thanks For Reading This Resource"

LinkedIn : [mirza-yasir-abdullah-Baig](#).